Hibernate ORM Mapping

Table of Contents

1)What is hibernate mapping ?....1
2)Basic understanding of O/R Mapping....2
3)Hibernate Mapping Types....5
4)Hibernate Sample Example....9
5)Compilation and Execution:....9
a) ManageEmployee.java....9
b) hibernate.cfg.xml....10
c) Employee.hbm.xml....11
Employee Table....12
6)Hibernate Mappings....12
a) Association Mappings:....12
7)One-to-One....13
a) TABLE STOCK....13
b) TABLE STOCK_DETAIL....13
Lets create following three xml files....19
Stock.hbm.xml....19
StockDetail.hbm.xml....19
hibernate.cfg.xml....19
c) Stock.hbm.xml....20
d) StockDetail.hbm.xml....21
f) TestOnetoOne.java....23
g) Output....24
8)One-to-Many....24
Lets create following three xml files....28
Address.hbm.xml....28
TCSEmployee.hbm.xml....28
hibernate.cfg.xml....28
b) TCSEmployee.hbm.xml....29
c) Hibernate.cfg.xml....30
d) TestOnetoMany.java....30
e) Output....31
9)Important Terms or Tags to be discussed....31
10)Many-to-Many....32
a) Student.java....34
Lets create following three xml files....35
TestManytoMany.java....37

1) What is hibernate mapping ?

An Object/relational mappings are usually defined in an XML document. This mapping file instructs Hibernate how to map the defined class or classes to the database tables.Though many Hibernate users choose to write the XML by hand, a number of tools exist to generate the mapping document. These include XDoclet, Middlegen and AndroMDA for advanced Hibernate users.

Hibernate mapping file is used to map the Pojo class and its attributes to corresponding table and its columns in the Data base.

2) Basic understanding of O/R Mapping

Lets consider below mentioned POJO Class Employee.

```
public class Employee {
    private int id;
    private String firstName;
    private String lastName;
    private int salary;
    public int getId() {
        return id;}
    public void setId(int id) {
        this.id = id;}

    public String getFirstName() {
        return firstName;}
    public void setFirstName(String first_name) {
        this.firstName = first_name;}

    public String getLastName() {
        return lastName;}
    public void setLastName(String last_name) {
        this.lastName = last_name;}

    public int getSalary() {
        return salary;}
    public void setSalary(int salary) {
        this.salary = salary;}
}
```

There would be one table corresponding to each object you are willing to provide persistence. Consider above objects need to be stored and retrieved into the following RDBMS table:

```
create table EMPLOYEE (
   id INT NOT NULL auto_increment,
   first_name VARCHAR(20) default NULL,
   last_name  VARCHAR(20) default NULL,
   salary     INT  default NULL,
   PRIMARY KEY (id)
);
```

Based on the two above entities we can define following mapping file which instructs Hibernate how to map the defined class or classes to the database tables.

a) <u>Employee.hbm.xml</u>

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
 "-//Hibernate/Hibernate Mapping DTD//EN"
 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
   <class name="Employee" table="EMPLOYEE">
      <meta attribute="class-description">
         This class contains the employee detail.
      </meta>
      <id name="id" type="int" column="id">
         <generator class="native"/>
      </id>
      <property name="firstName" column="first_name" type="string"/>
      <property name="lastName" column="last_name" type="string"/>
      <property name="salary" column="salary" type="int"/>
   </class>
</hibernate-mapping>
```

We should save the mapping document in a file with the format <pojo_classname>.hbm.xml. We saved our mapping document in the file Employee.hbm.xml. Let us see little detail about the mapping elements used in the mapping file:

1. The mapping document is an XML document having <hibernate-mapping> as the root element which contains all the <class> elements.
2. The <class> elements are used to define specific mappings from a Java classes to the database tables. The Java class name is specified using the name attribute of the class element and the database table name is specified using the table attribute.
3. The <meta> element is optional element and can be used to create the class description.
4. The <id> element maps the unique ID attribute in class to the primary key of the database table. The name attribute of the id element refers to the property in the class and the column attribute refers to the column in the database table. The type attribute holds the hibernate mapping type, this mapping types will convert from Java to SQL data type
5. The <generator> element within the id element is used to automatically generate the primary key values. Set the class attribute of the generator element is set to native to let hibernate pick

up either identity, sequence or hilo algorithm to create primary key depending upon the capabilities of the underlying database

6. The <property> element is used to map a Java class property to a column in the database table. The name attribute of the element refers to the property in the class and the column attribute refers to the column in the database table. The type attribute holds the hibernate mapping type, this mapping types will convert from Java to SQL data type

3) Hibernate Mapping Types

When we prepare a Hibernate mapping document, we have seen that we map Java data types into RDBMS data types. The types declared and used in the mapping files are not Java data types; they are not SQL database types either. These types are called Hibernate mapping types, which can translate from Java to SQL data types and vice versa.

Please find the below tables for more details

a) Primitive Data Types

Mapping type	Java type	ANSI SQL Type
integer	int or java.lang.Integer	INTEGER
long	long or java.lang.Long	BIGINT
short	short or java.lang.Short	SMALLINT
float	float or java.lang.Float	FLOAT
double	double or java.lang.Double	DOUBLE
big_decimal	java.math.BigDecimal	NUMERIC
character	java.lang.String	CHAR(1)
string	java.lang.String	VARCHAR
byte	byte or java.lang.Byte	TINYINT
boolean	boolean or java.lang.Boolean	BIT
yes/no	boolean or java.lang.Boolean	CHAR(1) ('Y' or 'N')
true/false	boolean or java.lang.Boolean	CHAR(1) ('T' or 'F')

b) Date/Time Types

Mapping type	Java type	ANSI SQL Type
date	java.util.Date or java.sql.Date	DATE
time	java.util.Date or java.sql.Time	TIME
timestamp	java.util.Date or java.sql.Timestamp	TIMESTAMP
calendar	java.util.Calendar	TIMESTAMP
calendar_date	java.util.Calendar	DATE

c) **Binary and Large Objects**

Mapping type	Java type	ANSI SQL Type
binary	byte[]	VARBINARY (or BLOB)
text	java.lang.String	CLOB
serializable	any Java class that implements java.io.Serializable	VARBINARY (or BLOB)
clob	java.sql.Clob	CLOB
blob	java.sql.Blob	BLOB

d) **JDK - RELATED TYPES**

Mapping type	Java type	ANSI SQL Type
class	java.lang.Class	VARCHAR
locale	java.util.Locale	VARCHAR
timezone	java.util.TimeZone	VARCHAR
currency	java.util.Currency	VARCHAR

4) Hibernate Sample Example

Let us try an example of using Hibernate to provide Java persistence in a standalone application. We will go through different steps involved in creating Java Application using Hibernate technology.

As per the previous example, we have already created a POJO Employee.java, its parallel DATABASE entity and the corresponding mapping of POJO vs Database in Employee.hbm.xml file.

Finally, we will create our application class with the main() method to run the application. We will use this application to save few Employee's records

5) Compilation and Execution:

Here are the steps to compile and run the above mentioned application. Make sure you have set PATH and CLASSPATH appropriately before proceeding for the compilation and execution.

1. Create hibernate.cfg.xml configuration file
2. Create Employee.hbm.xml mapping file as shown above
3. Create Employee.java source file as shown above and compile it.
4. Create ManageEmployee.java source file as shown below and compile it.
5. Execute ManageEmployee binary to run the program.

a) <u>ManageEmployee.java</u>

hibernate.cfg.xml | Employee.java | Employee.hbm.xml | ManageEmployee.java

```java
package pd.practice.hibernate.Dummy;
import org.hibernate.HibernateException;
public class ManageEmployee {
    public static void main(String[] args) {
        ManageEmployee ME = new ManageEmployee();
        Integer empID1 = ME.addEmployee("Zara", "Ali", 1000);
    }
    public Integer addEmployee(String fname, String lname, int salary) {
        SessionHelper sessionHelper = new SessionHelper();
        Session session = sessionHelper.getHibernateSession();
        Transaction transaction = null;
        Integer employeeID = null;
        try {
            transaction = session.beginTransaction();
            Employee employee = new Employee();
            employee.setFirstName(fname);
            employee.setLastName(lname);
            employee.setSalary(salary);
            employeeID = (Integer) session.save(employee);
            transaction.commit();
        } catch (HibernateException exception) {
            if (transaction != null)
                transaction.rollback();
            exception.printStackTrace();
        } finally {
            sessionHelper.closeHibernateSession(session);
        }
        return employeeID;
    }
}
```

b) hibernate.cfg.xml

*hibernate.cfg.xml | Employee.java

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property name="hibernate.connection.url">jdbc:oracle:thin:@172.26.132.40:1521:ORCLILP</property>
        <property name="hibernate.connection.username">a109core</property>
        <property name="hibernate.connection.password">a109core</property>
        <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="show_sql">true</property>
        <property name="hibernate.temp.use_jdbc_metadata_defaults">false</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="format_sql">true</property>
        <property name="use_sql_comments">true</property>
        <property name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>
        <property name="hibernate.cache.region.factory_class">
            org.hibernate.cache.ehcache.EhCacheRegionFactory
        </property>
        <mapping resource="Employee.hbm.xml" />
        <mapping class="pd.practice.hibernate.Dummy.Employee" />
</session-factory>
</hibernate-configuration>
```

c) Employee.hbm.xml

hibernate.cfg.xml | Employee.java | Employee.hbm.xml | ManageEmployee.java

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
 "-//Hibernate/Hibernate Mapping DTD//EN"
 "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">

<hibernate-mapping>
   <class name="pd.practice.hibernate.Dummy.Employee" table="EMPLOYEE">
       <meta attribute="class-description">
           This class contains the employee detail.
       </meta>
       <id name="id" type="int" column="id">
           <generator class="native"/>
       </id>
       <property name="firstName" column="first_name" type="string"/>
       <property name="lastName" column="last_name" type="string"/>
       <property name="salary" column="salary" type="int"/>
   </class>
</hibernate-mapping>
```

Once we compile and execute this program , following details would be inserted into the table.

```java
package pd.practice.hibernate.Dummy;
import org.hibernate.HibernateException;
public class ManageEmployee {
    public static void main(String[] args) {
        ManageEmployee ME = new ManageEmployee();
        Integer empID1 = ME.addEmployee("Zara", "Ali", 1000);
    }
    public Integer addEmployee(String fname, String lname, int salary) {
        SessionHelper sessionHelper = new SessionHelper();
        Session session = sessionHelper.getHibernateSession();
        Transaction transaction = null;
        Integer employeeID = null;
        try {
            transaction = session.beginTransaction();
            Employee employee = new Employee();
            employee.setFirstName(fname);
            employee.setLastName(lname);
            employee.setSalary(salary);
            employeeID = (Integer) session.save(employee);
            transaction.commit();
        } catch (HibernateException exception) {
            if (transaction != null)
                transaction.rollback();
            exception.printStackTrace();
        } finally {
            sessionHelper.closeHibernateSession(se
        }
        return employeeID;
    }
}
```

Employee Table

ID	FIRST_NAME	LAST_NAME	SALARY
NUMBER (10)	VARCHAR2 (1020)	VARCHAR2 (1020)	NUMBER (10)
1	Zara	Ali	1,000

6) Hibernate Mappings

a) Association Mappings:

The mapping of associations between entity classes and the relationships between tables is the soul of ORM. Following are the four ways in which the cardinality of the relationship between the objects can be expressed. An association mapping can be unidirectional as well as bidirectional.

Mapping type	Description
Many-to-One	Mapping many-to-one relationship using Hibernate
One-to-One	Mapping one-to-one relationship using Hibernate
One-to-Many	Mapping one-to-many relationship using Hibernate
Many-to-Many	Mapping many-to-many relationship using Hibernate

One-to-One

A one-to-one relationship table design, a STOCK table contains exactly one record in STOCK_DETAIL table. Both tables have the same Stock_Id as primary key. In STOCK_DETAIL table, Stock_Id is the primary key and also a foreign key to STOCK table. This is the common way of define "one-to-one" table relationship.

```
CREATE TABLE STOCK
(
   STOCK_ID          NUMBER(4) PRIMARY KEY,
   STOCK_CODE        VARCHAR2(4),
   STOCK_NAME        VARCHAR2(25)
)

CREATE TABLE STOCK_DETAIL
(
   STOCK_ID      NUMBER(4) REFERENCES STOCK(STOCK_ID),
   COMP_NAME     VARCHAR2(25),
   COMP_DESC     VARCHAR2(25),
   REMARK        VARCHAR2(25),
   LISTED_DATE   DATE
)
```

a) TABLE STOCK

STOCK_ID	STOCK_CODE	STOCK_NAME
NUMBER (4)	VARCHAR2 (4)	VARCHAR2 (25)

b) TABLE STOCK_DETAIL

STOCK_ID	COMP_NAME	COMP_DESC	REMARK	LISTED_DATE
NUMBER (4)	VARCHAR2 (25)	VARCHAR2 (25)	VARCHAR2 (25)	DATE

Lets create two model classes to represent above two tables. **Stock.java and StockDetail.java**

Stock.java

```java
package pd.practice.hibernate.OnetoOne;
import java.io.Serializable;
public class Stock implements Serializable {
    private Integer stockId;
    private String stockCode;
    private String stockName;
    private StockDetail stockDetail;

    public Integer getStockId() {
        return stockId;
    }
    public void setStockId(Integer stockId) {
        this.stockId = stockId;
    }
    public String getStockCode() {
        return stockCode;
    }
    public void setStockCode(String stockCode) {
        this.stockCode = stockCode;
    }
    public String getStockName() {
        return stockName;
    }
    public void setStockName(String stockName) {
        this.stockName = stockName;
    }
    public StockDetail getStockDetail() {
        return stockDetail;
    }
    public void setStockDetail(StockDetail stockDetail) {
        this.stockDetail = stockDetail;
    }
}
```

StockDetail.java

```java
import java.io.Serializable;
import java.util.Date;
public class StockDetail implements Serializable {
    private Integer stockId;
    private Stock stock;
    private String compName;
    private String compDesc;
    private String remark;
    private Date listedDate;
    public Integer getStockId() {
        return stockId;}
    public void setStockId(Integer stockId) {
        this.stockId = stockId;}
    public Stock getStock() {
        return stock;}
    public void setStock(Stock stock) {
        this.stock = stock;}
    public String getCompName() {
        return compName;}
    public void setCompName(String compName) {
        this.compName = compName;}
    public String getCompDesc() {
        return compDesc;}
    public void setCompDesc(String compDesc) {
        this.compDesc = compDesc;}
    public String getRemark() {
        return remark;}
    public void setRemark(String remark) {
        this.remark = remark;}
    public Date getListedDate() {
        return listedDate;}
    public void setListedDate(Date listedDate) {
        this.listedDate = listedDate;}
}
```

Lets create following three xml files.

- Stock.hbm.xml
- StockDetail.hbm.xml
- hibernate.cfg.xml

c) Stock.hbm.xml

Stock.hbm.xml | StockDetail.hbm.xml | hibernate.cfg.xml | TestOnetoOne.java | ManageEmployee.java | StockDetail.java

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="pd.practice.hibernate.OnetoOne.Stock" table="STOCK" >
        <id name="stockId" type="java.lang.Integer">
            <column name="STOCK_ID" />
            <!--<generator class="native"> <param
                name="sequence">stockId_seq</param>
            </generator>
            --></id>
        <property name="stockCode" type="string">
            <column name="STOCK_CODE" length="10" not-null="true" unique="true" />
        </property>
        <property name="stockName" type="string">
            <column name="STOCK_NAME" length="20" not-null="true" unique="true" />
        </property>
        <one-to-one name="stockDetail" class="pd.practice.hibernate.OnetoOne.StockDetail"
            cascade="save-update"></one-to-one>
    </class>
</hibernate-mapping>
```

d) StockDetail.hbm.xml

Stock.hbm.xml | StockDetail.hbm.xml | hibernate.cfg.xml

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://hibernate.sourceforge.net/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="pd.practice.hibernate.OnetoOne.StockDetail" table="STOCK_DETAIL">
        <id name="stockId" type="java.lang.Integer">
            <column name="STOCK_ID" />
            <generator class="foreign">
                <param name="property">stock</param>
            </generator>
        </id>
        <one-to-one name="stock" class="pd.practice.hibernate.HQLOnetoOne.Stock"
            constrained="true"></one-to-one>
        <property name="compName" type="string">
            <column name="COMP_NAME" length="100" not-null="true" />
        </property>
        <property name="compDesc" type="string">
            <column name="COMP_DESC" not-null="true" />
        </property>
        <property name="remark" type="string">
            <column name="REMARK" not-null="true" />
        </property>
        <property name="listedDate" type="date">
            <column name="LISTED_DATE" length="10" not-null="true" />
        </property>
    </class>
</hibernate-mapping>
```

e) <u>hibernate,cfg.xml</u>

Stock.hbm.xml | StockDetail.hbm.xml | hibernate.cfg.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property name="hibernate.connection.url">jdbc:oracle:thin:@172.26.132.40:1521:ORCLILP</property>
        <property name="hibernate.connection.username">a109core</property>
        <property name="hibernate.connection.password">a109core</property>
        <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="show_sql">true</property>
        <property name="hibernate.temp.use_jdbc_metadata_defaults">false</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="format_sql">true</property>
        <property name="use_sql_comments">true</property>
        <property name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>
        <property name="hibernate.cache.region.factory_class">
            org.hibernate.cache.ehcache.EhCacheRegionFactory
        </property>
        <mapping resource="Stock.hbm.xml" />
        <mapping class="pd.practice.hibernate.OnetoOne.Stock" />
        <mapping resource="StockDetail.hbm.xml" />
        <mapping class="pd.practice.hibernate.OnetoOne.StockDetail" />
</session-factory>
</hibernate-configuration>
```

f) TestOnetoOne.java

Stock.hbm.xml | StockDetail.hbm.xml | hibernate.cfg.xml | TestOnetoOne.java | ManageEmployee.java | StockDetail.java

```java
package pd.practice.hibernate.OnetoOne;
import java.util.Date;
public class TestOnetoOne {
    public static void main(String[] args) {
        SessionHelper sessionHelper = new SessionHelper();
        Session session = sessionHelper.getHibernateSession();
        session.beginTransaction();
        Stock stock = new Stock();
        stock.setStockId(new Integer("5555"));
        stock.setStockCode("4715");
        stock.setStockName("GENM");
        StockDetail stockDetail = new StockDetail();
        stockDetail.setCompName("GENTING MALAYSIA");
        stockDetail.setCompDesc("Best Resort in the World");
        stockDetail.setListedDate(new Date());
        stockDetail.setStock(stock);
        stockDetail.setRemark("BLUE CHIP");
        stock.setStockDetail(stockDetail);
        session.save(stock);
        session.getTransaction().commit();
        System.out.println("Done");
        sessionHelper.closeHibernateSession(session);
    }
```

g) <u>Output</u>

STOCK_ID	STOCK_CODE	STOCK_NAME
5,555	4715	GENM

STOCK_ID	COMP_NAME	COMP_DESC	REMARK	LISTED_DATE
5,555	GENTING MALAYSIA	Best Resort in the World	BLUE CHIP	5/22/2014 12:00:00 AM

7) One-to-Many

A One-to-Many mapping can be implemented using a Set java collection that does not contain any duplicate element. We already have seen how to map Set collection in hibernate, so if you already learned Set mapping then you are all set to go with one-to-many mapping. A Set is mapped with a <set> element in the mapping table and initialized with java.util.HashSet. You can use Set collection in your class when there is no duplicate element required in the collection.

Hibernate provides three types of collection

- **Bag**

  A Bag is a java collection that stores elements without caring about the sequencing but allow duplicate elements in the list. A bag is a random grouping of the objects in the list.A Collection is mapped with a <bag> element in the mapping table and initialized with java.util.ArrayList.

- **List**

  A List is a java collection that stores elements in sequence and allow duplicate elements. The user of this interface has precise control over where in the list each element is inserted. The user can access elements by their integer index, and search for elements in the list. More formally, lists typically allow pairs of elements e1 and e2 such that e1.equals(e2), and they typically allow multiple null elements if they allow null elements at all.A List is mapped with a <list> element in the mapping table and initialized with java.util.ArrayList.

- **Set**

  A Set is a java collection that does not contain any duplicate element. More formally, sets contain no pair of elements e1 and e2 such that e1.equals(e2), and at most one null element. So objects to be added to a set must implement both the equals() and hashCode() methods so that Java can determine whether any two elements/objects are identical. A Set is mapped with a <set> element in the mapping table and initialized with java.util.HashSet. You can use Set collection in your class when there is no duplicate element required in the collection

Lets discuss One-to-Many by mapping one address to many employees. Lets create two database tables as shown below.

```
CREATE TABLE EMP_ADDRESS
(
   ADDRESSID NUMBER (19)  PRIMARY KEY,
   STREET    VARCHAR2 (1020),
   CITY      VARCHAR2 (1020),
   STATE     VARCHAR2 (1020),
   COUNTRY   VARCHAR2 (1020)
)

CREATE TABLE TCS_EMPLOYEE
(
   EMPLOYEEID NUMBER (19) PRIMARY KEY,
   FIRSTNAME  VARCHAR2 (1020),
   LASTNAME   VARCHAR2 (1020),
   CELLPHONE  VARCHAR2 (1020),
   ADDRESSID  NUMBER (19) REFERENCES EMP_ADDRESS(ADDRESSID)
)
```

Lets create two model classes to represent above two tables. **Address.java and TCSEmployee.java**

Address.java | TCSEmployee.java

```java
package pd.practice.hibernate.OnetoMany;

import java.util.Set;

public class Address {
    private long addressId;
    private String street;
    private String city;
    private String state;
    private String country;
    private Set<TCSEmployee> employees;

    //getters and setters
}
```

Address.java | *TCSEmployee.java

```java
package pd.practice.hibernate.OnetoMany;

public class TCSEmployee {

    private long employeeId;
    private String firstname;
    private String lastname;
    private String cellphone;
    private Address address;
    public TCSEmployee(long employeeId, String firstname, String lastname,
            String cellphone) {
        //super();
        this.employeeId = employeeId;
        this.firstname = firstname;
        this.lastname = lastname;
        this.cellphone = cellphone;
    }

    //getters and setters

}
```

Lets create following three xml files.

- Address.hbm.xml
- TCSEmployee.hbm.xml
- hibernate.cfg.xml

a) <u>Address.hbm.xml</u>

Address.hbm.xml

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN" "http://www.hibernate.org
<hibernate-mapping>
    <class name="pd.practice.hibernate.OnetoMany.Address" table="EMP_ADDRESS">
        <id name="addressId" type="long" column="addressId">
                <generator class="sequence"> <param
                name="sequence">addressId_seq</param> </generator>
        </id>
        <property name="street">
            <column name="Street" />
        </property>
        <property name="city">
            <column name="City" />
        </property>
        <property name="state" >
            <column name="State" />
        </property>
        <property name="country">
            <column name="Country" />
        </property>
        <set name="employees" table="TCS_EMPLOYEE" inverse="true" lazy="true"
            fetch="select">
            <key><column name="addressId" not-null="true" /></key>
            <one-to-many class="pd.practice.hibernate.OnetoMany.TCSEmployee" />
        </set>
    </class>
</hibernate-mapping>
```

b) TCSEmployee.hbm.xml

TCSEmployee.hbm.xml

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="pd.practice.hibernate.OnetoMany.TCSEmployee"
        table="TCS_EMPLOYEE">
        <id name="employeeId" type="long" column="employeeId" >
        </id>
        <property name="firstname" >
            <column name="FirstName" />
        </property>
        <property name="lastname">
            <column name="LastName" />
        </property>
        <property name="cellphone">
            <column name="CellPhone" />
        </property>
        <many-to-one name="address" class="pd.practice.hibernate.OnetoMany.Address" fetch="select" >
            <column name="addressId" ></column>
        </many-to-one>
    </class>
</hibernate-mapping>
```

c) Hibernate.cfg.xml

hibernate.cfg.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property name="hibernate.connection.url">jdbc:oracle:thin:@172.26.132.40:1521:ORCLILP</property>
        <property name="hibernate.connection.username">a109core</property>
        <property name="hibernate.connection.password">a109core</property>
        <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="show_sql">true</property>
        <property name="hibernate.temp.use_jdbc_metadata_defaults">false</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="format_sql">true</property>
        <property name="use_sql_comments">true</property>
        <property name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>
        <property name="hibernate.cache.region.factory_class">
            org.hibernate.cache.ehcache.EhCacheRegionFactory
        </property>
        <mapping resource="TCSEmployee.hbm.xml" />
        <mapping class="pd.practice.hibernate.OnetoMany.TCSEmployee" />
        <mapping resource="Address.hbm.xml" />
        <mapping class="pd.practice.hibernate.OnetoMany.Address" />
</session-factory>
</hibernate-configuration>
```

d) TestOnetoMany.java

TestOnetoMany.java

```java
package pd.practice.hibernate.OnetoMany;
import org.hibernate.Session;
public class TestOnetoMany {
    public static void main(String[] args) {
        SessionHelper sessionHelper = new SessionHelper();
        Session session = sessionHelper.getHibernateSession();
        session.beginTransaction();
        Address address = new Address();
        address.setStreet("Lake Gardens");
        address.setCity("Kolkata");
        address.setState("West Bengal");
        address.setCountry("India");
        session.save(address);
        TCSEmployee e1 = new TCSEmployee(11, "Rakesh", "Sharma", "8238140053");
        TCSEmployee e2 = new TCSEmployee(12, "Kapil", "Kaushal", "8238140059");
        TCSEmployee e3 = new TCSEmployee(13, "Rakesh", "Sharma", "8238140060");
        TCSEmployee e4 = new TCSEmployee(14, "Kapil", "Kaushal", "8238140061");
        e1.setAddress(address);
        e2.setAddress(address);
        e3.setAddress(address);
        e4.setAddress(address);
        session.save(e1);
        session.save(e2);
        session.save(e3);
        session.save(e4);
        session.getTransaction().commit();
        sessionHelper.closeHibernateSession(session);
    }}
```

e) Output

ADDRESSID	STREET	CITY	STATE	COUNTRY
1	Lake Gardens	Kolkata	West Bengal	India

EMPLOYEEID	FIRSTNAME	LASTNAME	CELLPHONE	ADDRESSID
11	Rakesh	Sharma	8238140053	1
12	Kapil	Kaushal	8238140059	1
13	Rakesh	Sharma	8238140060	1
14	Kapil	Kaushal	8238140061	1

8) Important Terms or Tags to be discussed

Keyword	Explanation	Optional Values(Default)
inverse	The “inverse” keyword is always declare in one-to-many and many-to-many relationship (many-to-one doesn’t has inverse keyword), it means which side is responsible to take care of the relationship. In Hibernate, only the “relationship owner” should maintain the relationship, and the “inverse” keyword is created to defines which side is the owner to maintain the relationship. In short, inverse=”true” means this is the relationship owner, and inverse=”false” (default) means it’s not. As per the case mentioned above, inverse=”true” is mentioned in **employees set** in Address.hbm.xml file/ that means **TCS_EMPLOYEE** is the relationship owner and it will update the relationship	• True • False(default)
lazy	In the above configuration in **Address.hbm.xml** if **lazy="true"** means when you load the Employee object that time child object Address is not loaded. You need extra call to database to get address objects. If you call employee.getAdrress() then that time database query fires and return results. Fresh database call is happening. **Employee.hbm.xml** If **lazy="false"** means When you load the Employee object that time child Address object is also loaded and set to setAddresss() method. If you call employee.getAddress() then loaded data returns. No fresh database call is required	• True(default) • False
fetch	*fetch=”join”* : Disable the lazy loading, always load all	• join

	the collections and entities by using join mechanism. *fetch="select"*: Lazy load all the collections and entities.	• select(default)
key	Its one of the property through which mapping is connected. More over its Primary Key/ Foreign Key relationship column or joined column in both the table.	
generator	All the generator classes implements the org.hibernate.id.IdentifierGenerator interface. The application programmer may create one's own generator classes by implementing the IdentifierGenerator interface. Hibernate framework provides many built-in generator classes as explained in next column ***assigned*** It is the default generator strategy if there is no <generator> element . In this case, application assigns the id. ***Sequence*** It uses the sequence of the database. if there is no sequence defined, it creates a sequence automatically e.g. in case of Oracle database, it creates a sequence named HIBERNATE_SEQUENCE. ***Increment*** It generates the unique id only if no other process is inserting data into this table. It generates short, int or long type identifier. The first generated identifier is 1 normally and incremented as 1	• increment • assigned • sequence • hilo • native • identity • seqhilo • uuid • guid • select • foreign • sequence-identity
cascade	The "Cascade" keyword is often appear on the collection mapping to manage the state of the collection automatically. If we write cascade = "all" then changes at parent class object will be effected to child class object too, if we write cascade = "all" then all operations like insert, delete, update at parent object will be effected to child object also	• none(default) • save • save-update • update • delete • all • all-delete-orphan

9) Many-to-Many

A Many-to-Many mapping can be implemented using a Set java collection that does not contain any duplicate element. We already have seen how to map Set collection in hibernate, so if you already learned Set mapping then you are all set to go with many-to-many mapping.
A Set is mapped with a <set> element in the mapping table and initialized with java.util.HashSet. You can use Set collection in your class when there is no duplicate element required in the collection.

Lets take up many-to-Many in a form of student and course with the basic concepts of many students can opt for many courses.

```
CREATE TABLE STUDENT
(
   STUDENTID   NUMBER (19) PRIMARY KEY,
   STUDENTNAME VARCHAR2 (1020)
)
CREATE TABLE COURSE
(
   COURSEID   NUMBER (19) PRIMARY KEY,
   COURSENAME VARCHAR2 (1020)
)
CREATE TABLE STUDENT_COURSE
(
   COURSENAME  NUMBER (19) REFERENCES COURSE (COURSEID),
   STUDENTNAME NUMBER (19) REFERENCES STUDENT (STUDENTID),
   PRIMARY KEY (STUDENTNAME, COURSENAME)
)
```

Lets create Student.java and Course.java POJOs

a) **Student.java**

```java
package pd.practice.hibernate.HQLManytoMany;
import java.util.HashSet;
public class Student {
    private long studentId;
    private String studentName;
    private Set<Course> courses = new HashSet<Course>();
    public Student(String studentName) {
        //this.studentId = studentId;
        this.studentName = studentName;
    }
    public long getStudentId() {
        return studentId;
    }public void setStudentId(long studentId) {
        this.studentId = studentId;
    }
    public String getStudentName() {
        return studentName;
    }public void setStudentName(String studentName) {
        this.studentName = studentName;
    }
    public Set<Course> getCourses() {
        return courses;
    }public void setCourses(Set<Course> courses) {
        this.courses = courses;
    }
}
```

b) **Course.java**

```java
package pd.practice.hibernate.HQLManytoMany;
import java.util.HashSet;
public class Course {
    private long courseId;
    private String courseName;
    private Set<Student> students = new HashSet<Student>();
    public Course(String courseName) {
        //this.courseId = courseId;
        this.courseName = courseName;
    }
    public long getCourseId() {
        return courseId;
    }public void setCourseId(long courseId) {
        this.courseId = courseId;
    }
    public String getCourseName() {
        return courseName;
    }public void setCourseName(String courseName) {
        this.courseName = courseName;
    }
    public Set<Student> getStudents() {
        return students;
    }public void setStudents(Set<Student> students) {
        this.students = students;
    }
}
```

Lets create following three xml files.

- Student.hbm.xml
- Course.hbm.xml
- hibernate.cfg.xml

c) Student.hbm.xml

Student.hbm.xml

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="pd.practice.hibernate.HQLManytoMany.Student" table="Student">
        <id name="studentId" type="long" column="StudentId">
                <generator class="native"> <param
                name="sequence">studentId_seq</param> </generator>
        </id>
        <property name="studentName">
            <column name="StudentName" />
        </property>
        <set name="courses" table="Student_Course" inverse="false" lazy="true"
            fetch="select" cascade="all">
            <key column="StudentId" />
            <many-to-many column="CourseId" class="pd.practice.hibernate.HQLManytoMany.Course" />
        </set>
    </class>
</hibernate-mapping>
```

d) Course.hbm.xml

Course.hbm.xml

```xml
<?xml version="1.0"?>
<!DOCTYPE hibernate-mapping PUBLIC "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="pd.practice.hibernate.HQLManytoMany.Course" table="Course">
        <id name="courseId" type="long" column="CourseId">
                <generator class="native"> <param
                name="sequence">courseId_seq</param> </generator>
        </id>
        <property name="courseName">
            <column name="CourseName" />
        </property>
        <set name="students" table="Student_Course" inverse="false" lazy="true"
            fetch="select" cascade="all">
            <key column="CourseId" />
            <many-to-many column="StudentId" class="pd.practice.hibernate.HQLManytoMany.Student" />
        </set>
    </class>
</hibernate-mapping>
```

Note : There is a glue table in many-to-many mapping, this table acts as an intermediate table in between two mapped tables. Please note to keep the glue/joined table name same in both the XML Files E.g. Student_Course Table

hibernate.cfg.xml

```xml
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
"-//Hibernate/Hibernate Configuration DTD 3.0//EN"
"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.connection.driver_class">oracle.jdbc.driver.OracleDriver</property>
        <property name="hibernate.connection.url">jdbc:oracle:thin:@172.26.132.40:1521:ORCLILP</property>
        <property name="hibernate.connection.username">a109core</property>
        <property name="hibernate.connection.password">a109core</property>
        <property name="hibernate.dialect">org.hibernate.dialect.Oracle10gDialect</property>
        <property name="show_sql">true</property>
        <property name="hibernate.temp.use_jdbc_metadata_defaults">false</property>
        <property name="hbm2ddl.auto">create</property>
        <property name="format_sql">true</property>
        <property name="use_sql_comments">true</property>
        <property name="hibernate.cache.provider_class">org.hibernate.cache.EHCacheProvider</property>
        <property name="hibernate.cache.region.factory_class">
            org.hibernate.cache.ehcache.EhCacheRegionFactory
        </property>
        <mapping resource="Student.hbm.xml" />
        <mapping class="pd.practice.hibernate.HQLManytoMany.Student" />
        <mapping resource="Course.hbm.xml" />
        <mapping class="pd.practice.hibernate.HQLManytoMany.Course" />
</session-factory>
</hibernate-configuration>
```

TestManytoMany.java

```java
TestManytoMany.java
package pd.practice.hibernate.HQLManytoMany;
import org.hibernate.Session;
public class TestManytoMany {
    public static void main(String[] args) {
        SessionHelper sessionHelper = new SessionHelper();
        Session session = sessionHelper.getHibernateSession();
        System.out.println(session);
        session.beginTransaction();
        Course c1 = new Course("Operating System");
        Course c2 = new Course("Cryptography");
        Student s1 = new Student("Sachin Bharadwaj");
        Student s2 = new Student("Vikas Sharma");
        s1.getCourses().add(c1);
        s1.getCourses().add(c2);
        s2.getCourses().add(c1);
        s2.getCourses().add(c2);
        session.save(s1);
        session.save(s2);
        session.getTransaction().commit();
        sessionHelper.closeHibernateSession(session);
    }
}
```

d) <u>Output</u>

STUDENTID	STUDENTNAME
1	Sachin Bharadwaj
2	Vikas Sharma

COURSEID	COURSENAME
1	Cryptography
2	Operating System

STUDENTID	COURSEID
1	1
2	1
1	2
2	2